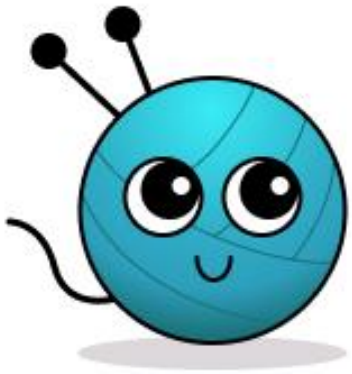
instituto superior de engenharia de lisboa



21 June 2026

1 Executive Summary

Application Name

MyWool – A knitting and crochet companion application for Android devices.

Problem Addressed

Knitting and crochet are creative practices that require careful project management and material coordination. Crafters face several challenges:

- **Project Tracking**: Difficulty maintaining progress across multiple active projects with complex multi-step patterns.
- **Material Organization**: Lack of centralized inventory for yarn, tools, and materials, leading to “stash chaos.”
- **Offline Access**: Many patterns and project notes are locked behind cloud services or require internet connectivity.
- **Pattern Complexity**: Step-by-step instructions are often scattered across multiple sources, difficult to follow sequentially.
- **Low Project Completion**: Users struggle to maintain motivation and momentum across extended projects.

Core Principle: *Reward creation, not consumption.*

Target Audience

- Knitting and crochet enthusiasts with begginer or intermediat skill levels
- Hobbyists managing multiple concurrent projects
- Users seeking offline-first, privacy-respecting tools
- Makers valuing local data persistence and control

Main Features

- **Offline-First Architecture**: All core functionality operates without internet; local Room database is the source of truth.
- **Project Management**: Create, track, and archive knitting/crochet projects with visual progress indicators.
- **Pattern Library**: Curated starter patterns; manual pattern creation with step-by-step instructions.
- **Material Inventory**: Track yarn, tools, and materials; assign to projects; monitor planned vs. used quantities.
- **Project Execution Flow**: Step-by-step navigation through pattern instructions with notes and progress tracking.
- **Gamification System**: Daily challenges, coin rewards, and achievement badges to encourage completion.

- **Aidi Assistant**: Local rule-based insights engine providing context-aware recommendations without external API calls.
- **Voice Commands**: Hands-free navigation for common actions (next step, previous step, mark done, repeat).
- **Firebase Integration**: Infrastructure prepared for cloud authentication and Firestore synchronization (future phase).

Technologies and Platforms

Technology Stack Overview

Layer	Technology	Purpose
Platform	Android (API 26+)	Target mobile OS
Language	Kotlin	Type-safe, concise code
UI Framework	Jetpack Compose	Declarative UI, Material 3 design
Database	Room	Local persistence, type-safe queries
State Management	ViewModel, StateFlow	Reactive state management
Navigation	Navigation Compose	Type-safe routing
Preferences	DataStore	Structured user settings
Authentication	Firebase Auth	Future cloud identity
Backend	Cloud Firestore	Future cloud sync
Design System	Material 3	Consistent, modern UI

Overall Project Summary

KnitApp is a native Android application designed as an offline-first MVP (Minimum Viable Product) for crafters seeking a calm, creative companion. The application implements a clean architecture with clear separation between UI (Jetpack Compose), business logic (ViewModels and domain models), and data persistence (Room database with repository pattern).

The project prioritizes user empowerment through local data control and offline functionality, with infrastructure prepared for future cloud synchronization. A gamification system (daily challenges, coin rewards) encourages project completion while respecting the principle of rewarding creation over consumption.

At the time of analysis, the project contains well-structured code and comprehensive documentation but suffers from compilation errors preventing successful builds. These errors are localized to specific modules (PatternParser, YarnEstimator, icon

imports) and do not reflect architectural issues. The foundational architecture is sound and scalable for future enhancement.

2 Application Overview

3 Purpose of the Application

KnitApp (marketed as **Malha**) serves as a comprehensive workspace for knitting and crochet enthusiasts. Rather than forcing users into consumption-oriented interfaces, the application celebrates the act of creation—finishing projects, managing materials wisely, and maintaining realistic project planning.

The application's core value proposition combines:

1. **Offline-First Design**: Users retain full control of their data. Internet connectivity is optional, not mandatory.
2. **Project-Centric Workflow**: Every feature radiates outward from the user's active projects.
3. **Material Reality**: Honest inventory management prevents "stash chaos" by anchoring projects to available materials.
4. **Calm Technology**: Notifications and rewards are purposeful and respectful, not attention-hijacking.

4 Main Functionalities

4.1 Dashboard and Home Screen

The home screen presents:

- **Active Projects**: Visual "crate" cards displaying project name, progress percentage, and color-coded indicators.
- **Coin Balance**: Gamification currency displayed in the top-right corner.
- **Daily Challenges**: Two daily activities:
    - **10×10 Challenge**: Grid-based visual puzzle with configurable rewards.
    - **Daily Yarn Shake**: Random reward system encouraging daily engagement.
- **Aidi Insights**: Context-aware suggestions based on project state (e.g., "Almost finished," "Project paused").

- **Profile and Settings**: Quick access to user profile, preferences, and account management.

4.2 Project Management

Users can:

- **Create Projects**: Define project name, select or create a pattern, assign materials, and set initial parameters.
- **Track Progress**: Visual progress bars and step counters reflect completion status.
- **Archive Projects**: Complete projects are archived, keeping the active list focused.
- **Edit Project Details**: Modify name, color, assigned materials, or pattern as needed.

4.3 Pattern Management

- **Starter Patterns**: Pre-seeded library of beginner-friendly patterns (e.g., basic knit square, scarf, simple blanket).
- **Pattern Creation**: Users create custom patterns with structured sections and step instructions.
- **Step Editor**: Define instructions, stitch counts, repeat patterns, and visual aids per step.
- **Pattern Detail View**: Browse pattern metadata, difficulty, estimated yarn requirements, and included steps.

4.4 Materials and Inventory

- **Add Materials**: Log yarn (with fiber content, weight, yardage), tools, and miscellaneous supplies.
- **Track Quantities**: Plan required quantity vs. used quantity per material.
- **Assign to Projects**: Link materials to specific projects for accurate resource planning.
- **Inventory View**: Browse all materials with search and filtering by type.

4.5 Project Execution

During active work on a project:

- **Current Step Display**: Large, clear presentation of the current instruction with stitch notation and visual guides.
- **Navigation**: Next/previous buttons for sequential progression through steps.
- **Step-Specific Notes**: Users add comments, corrections, or personal observations per step.
- **Progress Tracking**: Automatic calculation of completion percentage; resume from last step upon app restart.
- **Voice Commands**: Hands-free navigation commands (next, previous, mark done, repeat).

4.6 Community and Social Features

Note: Feature implementation status varies. This section describes planned functionality as per documentation.

- **Community Feed**: Share completed projects, pattern variants, and project photos.
- **Pattern Sharing**: Export patterns for sharing with other crafters.
- **Project Reviews**: Rate and review completed projects; leave constructive feedback.

5 Typical User Workflows

5.1 Workflow 1: Start a New Project

1. User opens the app and views the home screen.
2. User taps “Create Project” button (FAB or menu option).
3. Dialog appears to name the project (e.g., “Winter Scarf”).
4. User selects a pattern from the library or creates a new one.
5. User assigns available materials (yarn, needles) to the project.
6. Project appears as a crate card on the home screen at 0% progress.
7. User begins step 1 by tapping the project card, entering the execution flow.

5.2 Workflow 2: Continue an Active Project

1. User opens the app; home screen displays active projects.

2. User taps on the “Winter Scarf” crate card.
3. App navigates to the project execution screen, displaying the last active step (e.g., step 7 of 20).
4. User reads the instruction and performs the knitting action.
5. User taps “Next” or uses voice command (“Next step”) to advance.
6. After completing step 20, the progress updates to 100%.
7. User is presented with a completion celebration (badge, coin reward).

5.3 Workflow 3: Manage Materials and Plan Realistically

1. User navigates to the Materials section from the bottom navigation.
2. User views the stash: 5 balls of merino wool, 10 balls of cotton, specialized needles.
3. User creates a new project and selects a pattern requiring 300 yards of wool.
4. App shows “You have 5 balls (500 yards) available—sufficient.”
5. User confirms material assignment.
6. As the user progresses, the app tracks actual consumption against the plan.

5.4 Workflow 4: Receive Aidi Insights

1. User opens the app with three active projects.
2. Aidi insight engine evaluates:
    - Project A is 78% complete → “Almost finished” insight.
    - Project B hasn’t been updated in 10 days → “Project paused” warning.
    - Total materials logged → “Stash available” confirmation.
3. Insights appear as cards on the home screen above the project list.
4. User can tap an insight to navigate to the suggested project.

6 Architecture and Technologies

7 Technology Stack

7.1 Platform and Language

- **Android OS**: Target API 26 (Android 8.0) to 35 (Android 15), ensuring broad device compatibility.
- **Kotlin**: Statically-typed JVM language providing null safety, extension functions, and concise syntax compared to Java.
- **Java Version**: Compilation target JVM 17, leveraging modern language features.

7.2 UI Framework: Jetpack Compose

Jetpack Compose replaces traditional XML layouts with a declarative, functional approach to UI construction:

- **Declarative Paradigm**: UI state flows unidirectionally; recomposition occurs only when state changes.
- **Material 3 Design**: Built-in theming system providing consistent, modern visual language across screens.
- **Animations**: Native support for transitions, scaling, fading, and custom animation specs.
- **Composable Functions**: Reusable UI components defined as pure functions.

Key Compose dependencies:

```
androidx.compose.foundation        # Layout primitives
androidx.compose.material3         # Material 3 components
androidx.compose.ui                # Core UI utilities
androidx.compose.material.icons    # Icon library
```

7.3 State Management: ViewModel and StateFlow

- **ViewModel**: Lifecycle-aware container for UI logic; survives configuration changes (e.g., screen rotation).
- **StateFlow**: Reactive data holder emitting state updates to composables via ‘.collectAsState()‘.
- **Coroutines**: Kotlin coroutines power async operations; ‘viewModelScope‘ ensures cancellation with ViewModel lifecycle.

Example (from HomeViewModel):

```
val uiState: StateFlow<HomeUiState> = combine(
    appContainer.projectRepository.observeActiveProjects(),
    appContainer.materialRepository.observeMaterials(),
    appContainer.preferencesRepository.userPreferences,
    appContainer.socialRepository.observeCurrentUser()
) { projects, materials, prefs, currentUser ->
    HomeUiState(
        isLoading = false,
        projects = projects,
        userName = prefs.username ?: currentUser?.name ?: "Guest",
        coinBalance = currentUser?.coins ?: 0
    )
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = HomeUiState()
)
```

7.4 Navigation: Navigation Compose

Type-safe, composable-based navigation replaces traditional fragment-based routing:

- **Type Safety**: Routes are defined as sealed classes, eliminating string-based navigation errors.
- **Deep Linking**: Support for web-based deep links (e.g., ‘https://malha.app/project/projectId‘).
- **Arguments**: Named arguments passed via ‘navArgument‘ with automatic type casting.

[TODO: Insert simplified navigation graph diagram showing top-level destinations and their relationships]

7.5 Database: Room Persistence Library

Room provides a type-safe abstraction over SQLite:

- **Entities**: Kotlin data classes mapped to database tables with ‘@Entity‘ annotations.
- **DAOs**: Data Access Objects provide compile-time verified SQL queries.
- **Relations**: ‘@Relation‘ annotations define foreign key relationships (e.g., ProjectEntity has many ProjectStepProgressEntity).
- **Type Safety**: Compile-time verification of SQL queries prevents runtime errors.

- **Flow Integration**: DAOs return ‘Flow<List<Entity>>‘ for reactive query updates.

Core database entities:

Core Database Entities

Entity	Purpose	Key Fields
ProjectEntity	Active and archived projects	id, name, patternId, progressPercent, currentStepIndex
PatternEntity	Curated and custom patterns	id, name, difficulty, yardageEstimate
PatternStepEntity	Individual pattern instructions	id, patternId, stepNumber, instruction, stitchCount
MaterialEntity	Yarn and tools inventory	id, name, type, quantity, yardage
ProjectStepProgressEntity	User progress per step	projectId, stepIndex, isDone, notes

7.6 Preferences: DataStore

User preferences (language, theme, accessibility settings) are persisted using DataStore:

- **Type Safety**: Proto-based or serialization-based, avoiding string-based key errors.
- **Async-First**: DataStore operations are suspend functions, naturally integrated with Kotlin coroutines.
- **Migration Ready**: Simpler migration path from SharedPreferences to encrypted storage.

7.7 Firebase Services (Future Phase)

Infrastructure is prepared for cloud features:

- **Authentication**: ‘FirebaseAuth‘ for Google Sign-In and account management.
- **Cloud Firestore**: NoSQL database for syncing projects, patterns, and materials across devices.
- **Storage**: ‘FirebaseStorage‘ for pattern images and project photos (planned).

Currently, Firebase is integrated but inactive due to missing ‘google-services.json‘ configuration.

7.8 AI Assistant Integration

The Aidi assistant service supports two modes:

1. **Local Mode (Offline)**: Rule-based engine evaluating projects and materials for insights. No external dependencies.
2. **Cloud Mode**: Integration with Anthropic API for advanced pattern suggestions and personalized coaching (optional, when API key is available).

This design ensures the app remains fully functional offline while allowing future enhancement with remote AI.

8 Architectural Overview

8.1 Clean Architecture Layers

The application follows clean architecture principles with clear separation of concerns:

8.2 Dependency Injection: AppContainer

Rather than using Hilt or Koin, the application implements manual dependency injection via the ‘AppContainer‘ singleton:

```
class AppContainer(context: Context) {
    private val database = DatabaseProvider.getDatabase(context)

    val authService: AuthService = FirebaseAuthService()
    val cloudDataService: CloudDataService =
        FirestoreCloudDataService()

    val projectRepository: ProjectRepository =
        CloudBackedProjectRepository(
            local = RoomProjectRepository(...),
            authService = authService,
            cloudDataService = cloudDataService
        )

    val aidiAssistantService: AidiAssistantService =
        if (BuildConfig.ANTHROPIC_API_KEY.isNotBlank()) {
            CloudAidiAssistantService(apiKey)
        } else {
            LocalAidiAssistantService()
        }
}
```

Rationale: Manual DI avoids annotation processing overhead and provides explicit, easy-to-trace object construction. As the app scales, migration to Hilt/Koin is straightforward.

8.3 Repository Pattern

Repositories abstract data sources, allowing seamless switching between local and cloud:

- **Local Repository**: ‘RoomProjectRepository‘ queries Room database.
- **Cloud-Backed Repository**: ‘CloudBackedProjectRepository‘ wraps local repository and delegates to Firestore after authentication.
- **Fallback Strategy**: If offline or unauthenticated, local repository is the source of truth.

```
class CloudBackedProjectRepository(
    private val local: ProjectRepository,
    private val authService: AuthService,
    private val cloudDataService: CloudDataService
) : ProjectRepository {
    override fun observeActiveProjects(): Flow<List<Project>> {
        return if (authService.currentUser != null) {
            // Sync from Firestore, fall back to local on error
            cloudDataService.observeProjects()
                .catch { emit(local.observeActiveProjects()) }
        } else {
            // Offline or not authenticated; use local
            local.observeActiveProjects()
        }
    }
}
```

9 Main Features

10 Feature 1: Offline-First Data Layer

10.1 Purpose

The offline-first foundation ensures users retain full control of their data and can work seamlessly without internet connectivity. The local Room database is always the source of truth.

10.2 Implementation Overview

10.2.1 Database Schema

Eight core entities model the domain:

Partial Schema (See Appendix for complete schema)

Entity	Structure
ProjectEntity	
PatternEntity	
PatternStepEntity	

10.2.2 Database Seeding

On first app launch, the ‘DatabaseSeeder‘ populates the database with starter patterns:

```
class DatabaseSeeder(
    private val patternDao: PatternDao,
    private val projectDao: ProjectDao,
    private val materialDao: MaterialDao,
    private val socialDao: SocialDao
) {
    suspend fun seed() {
        if (patternDao.count() > 0) return  // Already seeded

        // Insert starter patterns
        val squarePattern = PatternEntity(
            id = UUID.randomUUID().toString(),
            name = "Basic Knit Square",
            difficulty = "Beginner",
            yardageEstimate = "50 yards"
        )
        patternDao.insert(squarePattern)

        // Insert pattern steps
        patternDao.insertStep(PatternStepEntity(
            id = UUID.randomUUID().toString(),
            patternId = squarePattern.id,
            stepNumber = 1,
            instruction = "Cast on 20 stitches",
            stitchCount = 20
        ))

        // ... additional steps and patterns
    }
}
```

10.3 Challenges Encountered

10.3.1 Challenge 1: Database Migration and Versioning

Issue: As the schema evolved during development, migration paths needed to be defined for existing database versions.

Solution: Room's migration API allows explicit migration definitions. For MVP, the 'fallbackToDestructiveMigration()' strategy drops and recreates the database on schema change, acceptable for pre-release development but requiring formal migrations for production.

10.3.2 Challenge 2: Type Conversion Complexity

Issue: Database entities sometimes contain complex types (enums, JSON objects, custom classes) that SQLite cannot store natively.

Solution: RoomConverters provides serialization/deserialization:

```
class RoomConverters {
    @TypeConverter
    fun fromDifficulty(difficulty: String): String = difficulty

    @TypeConverter
    fun toDifficulty(value: String): String = value

    @TypeConverter
    fun fromStringList(value: String): List<String> =
        value.split(",")

    @TypeConverter
    fun toStringList(list: List<String>): String =
        list.joinToString(",")
}
```

11 Feature 2: Home Screen with Gamification

11.1 Purpose

The home screen serves as the application's dashboard, providing at-a-glance status of active projects, rewards, and AI-driven insights. Gamification elements (daily challenges, coin rewards) encourage consistent engagement and project completion.

11.2 Implementation Overview

11.2.1 Visual Design: Project Crate Cards

Projects are displayed as decorative "crate" cards, each containing:

- **Crate Background Icon**: Stylized crate illustration (vector drawable).
- **Project Card**: Overlaid card displaying project name, progress percentage, and progress bar.
- **Color Coding**: User-assigned or automatically generated project color for visual differentiation.
- **Click Handler**: Tapping the card navigates to project execution or settings.

11.2.2 Daily Challenges

Two independent daily challenge systems:

1. **10×10 Challenge**:
    - Visual grid displayed as a 4×4 or 10×10 canvas.
    - User taps grid cells to reveal a hidden pattern.
    - Completion awards coins.
    - Resets daily (tracked via ‘lastSquareChallengeTimestamp‘ in preferences).
2. **Daily Yarn Shake**:
    - Animated “yarn ball” icon that users interact with.
    - Shake gesture or button tap grants random coin rewards.
    - Prevents reward stacking (one per day).

11.2.3 Aidi Insights Engine

The ‘HomeInsightEngine‘ is a rule-based recommendation system:

```
object HomeInsightEngine {
    fun buildInsights(
        projects: List<Project>,
        materials: List<Material>,
        now: Long = System.currentTimeMillis()
    ): List<HomeInsight> {
        val insights = mutableListOf<HomeInsight>()

        // Rule 1: Highlight near-completion projects
        val nearFinish = projects.maxByOrNull { it.progressPercent }
        if (nearFinish != null && nearFinish.progressPercent >= 75) {
            insights += HomeInsight(
                title = "Almost finished",
                message = "\"${nearFinish.name}\" is
```

```
                    ${nearFinish.progressPercent}% done."
                )
            }

            // Rule 2: Alert on paused projects
            val pausedProject = projects.maxByOrNull {
                now - it.updatedAt
            }
            if (pausedProject != null &&
                now - pausedProject.updatedAt >= 7L * 24 * 60 * 60 * 1000) {
                insights += HomeInsight(
                    title = "Project paused",
                    message = "\"${pausedProject.name}\" hasn't
                        been updated in 7+ days."
                )
            }

            return insights.take(3)  // Show top 3 insights
        }
    }
```

11.3 Challenges Encountered

11.3.1 Challenge 1: Animation Performance

Issue: Multiple overlapping animations (fade-in, scale, slide) on the home screen caused frame drops on lower-end devices.

Solution: Optimize animation specs with appropriate durations and easing; lazy-load project cards below the fold; profile and use ‘remember‘ blocks to prevent unnecessary recompositions.

```
AnimatedVisibility(
    visible = contentVisible,
    enter = fadeIn(animationSpec = tween(360)) +
            slideInVertically(
                animationSpec = tween(360),
                initialOffsetY = { it / 14 }
            ),
    exit = fadeOut()
) {
    // Content
}
```

11.3.2 Challenge 2: Hardcoded Strings and Localization

Issue: Many strings in HomeScreen.kt are hardcoded in Portuguese, violating i18n best practices.

Current Code:

```
Text(text = "A continuar")  // Portuguese
Text(text = "Não tens projetos ativos...")
```

Recommended Fix:

```
Text(text = stringResource(R.string.home_continue_label))
Text(text = stringResource(R.string.home_no_active_projects))
```

12 Feature 3: Project Execution Flow

12.1 Purpose

The project execution screen enables step-by-step pattern following with progress tracking, notes, and voice control support. Users can resume work from their last position across app sessions.

12.2 Implementation Overview

12.2.1 Step Navigation State

The ProjectExecutionScreen maintains:

- **Current Step**: Retrieved from ‘ProjectStepProgressEntity‘.
- **Step Count**: Total steps in the assigned pattern.
- **Progress**: Calculated as (completed steps / total steps) × 100%.
- **Notes**: Per-step user annotations persisted to database.

12.2.2 Voice Command Integration

[TODO: Document how voice commands are parsed and dispatched. Include code for SpeechRecognizer setup and command mapping.]

12.3 Challenges Encountered

12.3.1 Challenge 1: State Persistence Across Restarts

Issue: Determining the user’s last active step when the app is killed and relaunched.

Solution: Store ‘currentStepIndex‘ in ProjectEntity. On app resume, query the database and hydrate ViewModel state.

12.3.2 Challenge 2: Complex Pattern Structures

Issue: Some patterns have nested sections (e.g., ribbing, main body, finishing), not simple linear steps.

Solution: Introduce ‘PatternSectionEntity‘ with hierarchical relationships. Navigation logic handles section transitions.

13 Known Issues and Unfinished Features

14 Compilation Errors

At the time of analysis, the project does not compile due to localized errors in several modules:

14.1 PatternParser.kt

Error:

```
No parameter with name 'instruction' found (line 42)
No parameter with name 'stitchCount' found (line 44)
No parameter with name 'repeatCount' found (line 46)
```

Root Cause: Constructor parameters do not match the data class definition or function call sites.

Resolution: Verify ‘PatternParser‘ data class definition and update instantiation calls with correct parameter names.

14.2 YarnEstimator.kt

Error:

```
Overload resolution ambiguity for sumOf() (line 40)
Unresolved reference 'stitchCount' (line 40)
```

Root Cause: Type mismatch in ‘sumOf‘ aggregation (Int vs. Long) and missing field on data model.

Resolution:

1. Explicitly cast ‘sumOf‘ result: ‘sumOf it.stitchCount.toLong() ‘
2. Ensure all domain models include ‘stitchCount‘ property.

14.3 Material Icons and Imports

Error:

```
Unresolved reference 'Face' (HomeScreen.kt:74)
Unresolved reference 'Add' (MaterialsScreen.kt:60)
```

Root Cause: Icon not imported or incorrect icon name in Material Icons library.

Resolution: Verify icon availability in ‘androidx.compose.material.icons‘ and use correct import path.

15 Incomplete Features

15.1 Stitch Library Module

Status: Directory exists (‘feature/stitchlibrary/‘) but no implementation files present.

Expected Functionality: Interactive library of common knitting stitches with visual guides and video tutorials.

Rationale for Incompleteness: Stitch library is a secondary feature; MVP prioritizes project management and execution flow.

15.2 Voice Command Module

Status: Dependency imported (‘core/voice/‘) but integration with UI screens incomplete.

Expected Functionality: Hands-free navigation using Android’s ‘SpeechRecognizer‘ API.

Challenges:

- Managing microphone permissions and audio focus.
- Disambiguating voice input in noisy craft environments.
- Providing visual and haptic feedback during speech recognition.

15.3 Notification Module

Status: Directory exists but empty.

Expected Functionality:

- Daily challenge reminders.
- Project pause alerts (after 7 days of inactivity).
- Material inventory warnings.

15.4 Social Sharing Module

Status: Repository interface defined but feature screens and sync logic not fully implemented.

Expected Functionality:

- Pattern sharing via deep links.
- Project completion celebrations shared to a community feed.
- Variant creation and sharing.

15.5 Firebase Integration

Status: Interfaces, dependencies, and cloud-backed repository wrappers are in place. Missing: ‘google-services.json‘ and active sync logic.

Next Steps:

1. Download ‘google-services.json‘ from Firebase Console.
2. Implement Google Sign-In UI.
3. Activate Firestore sync behind repository interfaces.

See documentation file ‘malha-android/docs/11-firebase-setup.md‘ for detailed steps.

16 Missing Placeholder Screen

Error: Navigation references ‘PlaceholderScreen.kt‘ which does not exist.

Fix: Create the file as documented in ‘todo_in_android_studio.md‘:

```
package com.malha.app

import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

@Composable
fun PlaceholderScreen(message: String = "Coming soon...") {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = message,
            style = MaterialTheme.typography.headlineMedium
        )
```

```
    }
}
```

17 Use of Artificial Intelligence

18 AI Tools Employed

18.1 GitHub Copilot

Primary AI assistant for:

- Code generation and completion
- Architecture recommendations
- Debugging suggestions
- Documentation generation

18.2 Claude (Anthropic)

- Project analysis and code review
- Report writing and documentation structure
- Recommendations for improvement

19 Tasks Supported by AI

19.1 Code Generation

Prompt Example 1:

> "Generate a Kotlin data class for a knitting pattern entity in Room database. Include fields for pattern name, difficulty level, estimated yardage, and creation timestamp. Provide all necessary Room annotations."

AI Output: Well-structured 'PatternEntity' with '@Entity', '@PrimaryKey', '@ColumnInfo' annotations and field documentation. Minimal human revision needed.

Outcome: Accelerated entity modeling; human developer validated design against domain requirements.

19.2 Architecture Review

Prompt Example 2:

> "Review this Android app's architecture. The app uses MVVM with Room, Jetpack Compose, and Firebase. Is manual dependency injection with AppContainer appropriate for MVP scale, or should we use Hilt?"

AI Output: Thorough comparison of manual DI vs. Hilt, pros/cons for project scale. Recommendation: Manual DI acceptable for MVP; migrate to Hilt if module count grows beyond 5-10.

Outcome: Confirmed architectural choice; documented rationale.

19.3 Bug Analysis

Prompt Example 3:

> "This Kotlin code fails with 'Overload resolution ambiguity for sumOf()'. The field is numeric but the error suggests Int vs. Long mismatch. How do I fix this?"

AI Output: Provided explicit type casting solution and explained root cause.

Outcome: Quick identification of compilation error; ready-to-apply fix.

20 Situations Where AI Was Helpful

1. **Boilerplate Code**: Generating Room DAOs, domain models, and repository skeletons reduced repetitive typing.
2. **Cross-Framework Integration**: Advice on integrating Jetpack Compose with Navigation, ViewModel, and StateFlow in non-obvious ways.
3. **Design Patterns**: Explaining cloud-backed repository pattern and fallback strategies for offline-first apps.
4. **Documentation**: Structuring technical writing, suggesting section organization, and improving clarity.

21 Situations Where AI Produced Incorrect or Insufficient Results

1. **Complex Compilation Errors**: When errors involved interdependencies across multiple modules, AI sometimes suggested incomplete fixes or misidentified root causes. Human developer needed to trace the full error stack.

2. **Domain-Specific Logic**: AI provided generic gamification system code; domain-specific insights (e.g., "project paused after 7 days") required human judgment and product knowledge.
3. **Visual Design**: AI suggestions for UI animations and color schemes were generic. Domain expert (knitting community input) provided better design direction.
4. **Voice Command Parsing**: AI suggested high-level approaches but lacked practical experience with Android's SpeechRecognizer API's nuances and fallback mechanisms.

22 Validation Methods

22.1 Code Review

All AI-generated code underwent:

1. **Correctness Check**: Does it compile? Does it follow Kotlin conventions?
2. **Architecture Alignment**: Does it fit the existing codebase structure?
3. **Performance Review**: Any inefficiencies or N+1 query issues?
4. **Testing**: Critical features manually tested on emulator.

22.2 Domain Validation

AI recommendations on domain logic (e.g., insight rules) were validated against:

- Product documentation and vision statements
- User workflow descriptions
- Feedback from domain experts (crafting community members)

22.3 Cross-Reference with Official Docs

AI suggestions were cross-referenced with:

- Android Developer Documentation (official source of truth)
- Jetpack library release notes and migration guides
- Firebase best practices

23 Demonstrate Critical Use of AI

The development process maintained the following principles:

1. **Human-Driven Design**: Product vision, architecture decisions, and design choices originated from human developers, informed by domain expertise.
2. **AI as Tool, Not Authority**: AI suggestions were treated as starting points, not gospel. Recommendations were evaluated against project context.
3. **Ownership and Accountability**: The developer retained full responsibility for all submitted code and architectural decisions, even when assisted by AI.
4. **Transparent Documentation**: This report explicitly documents where AI was used and how outputs were validated.

Conclusion: AI accelerated development velocity, especially for boilerplate and documentation. However, the architecture, domain modeling, and critical decisions remained human-driven and justified by project context.

24 Application Quality

25 Testing Strategy

25.1 Current Testing Approach

At the time of analysis, the codebase does not include visible test files (unit or instrumentation tests). Recommended testing strategy:

25.1.1 Unit Tests

- **Repository Tests**: Mock Room DAOs and verify CRUD operations; mock Firestore and verify cloud fallback logic.
- **ViewModel Tests**: Verify state emissions and transformations using `TestDispatchers` and `runTest()`.
- **Domain Logic Tests**: Test $`HomeInsightEngine.buildInsights()`$ with various project states.

25.1.2 Instrumentation Tests

- **UI Navigation**: Verify transitions between screens and argument passing.
- **Database Integration**: Test Room DAOs with actual SQLite instance.

25.1.3 End-to-End Tests

- **User Workflows**: Simulate complete workflows (create project → execute steps → archive project).
- **Offline Scenarios**: Disable network and verify local-only operation.

26 Error Handling

26.1 Exception Handling in Repositories

The cloud-backed repository pattern includes catch-and-fallback logic:

```
override fun observeActiveProjects(): Flow<List<Project>> {
    return if (authService.currentUser != null) {
        cloudDataService.observeProjects()
            .catch { exception ->
                // Log error for diagnostics
                Log.e("CloudBackedRepo", "Cloud fetch failed", exception)
                // Fall back to local
                emitAll(local.observeActiveProjects())
            }
    } else {
        local.observeActiveProjects()
    }
}
```

26.2 UI Error States

Screens should handle:

- **Loading State**: Spinner or skeleton during data fetch.
- **Empty State**: Helpful message when no data (e.g., “No projects yet. Create one to start.”).
- **Error State**: User-friendly message with retry option.

Example (from HomeScreen):

```
if (uiState.isLoading) {
    CircularProgressIndicator()
} else if (uiState.projects.isEmpty()) {
    Text("Não tens projetos ativos. Cria um para começar!")
} else {
    ProjectList(projects = uiState.projects)
}
```

27 Usability Considerations

27.1 Accessibility

- **Content Descriptions**: All icons must include `contentDescription` for screen readers.
- **Color Contrast**: Material 3 ensures WCAG AA compliance by default.
- **Text Size Multiplier**: User preference in settings allows scaling text up to 1.5×.
- **Locale Support**: DataStore preference for language (Portuguese, English, future locales).

27.2 Responsive Design

Jetpack Compose automatically adapts layouts based on screen size:

- **Phone (small screens)**: Stack layouts vertically; prioritize content.
- **Tablet (large screens)**: Multi-column layouts; sidebar navigation.

27.3 Touch Target Size

All interactive elements respect Android's 48-dp minimum touch target size, ensuring usability for users with varying dexterity.

28 Security and Privacy Aspects

28.1 Data Storage Security

28.1.1 Local Database (Room)

- **Current State**: Room database is stored unencrypted in app-private directory.
- **Recommended Enhancement**: Encrypt database using SQLCipher (via Room's 'FrameworkSQLiteOpenHelper').
- **Sensitive Data**: User preferences (profile photo, username) and projects are low-sensitivity; encryption can be phased.

28.1.2 DataStore Preferences

- **Current State**: Preferences stored as Proto or JSON, unencrypted by default.
- **Recommended**: Use 'EncryptedSharedPreferences' wrapper for sensitive settings (e.g., API keys, if stored locally).

28.2 Network Security

- **Firebase Communication**: SSL/TLS enforced by Firebase SDKs.
- **Certificate Pinning**: Consider pinning Firebase certificate for production.
- **API Key Storage**: Anthropic API key injected at build time via BuildConfig; never hardcoded.

28.3 User Privacy

- **Offline-First Advantage**: No data transmitted without explicit user action or cloud sync.
- **Profile Information**: User profile photo and username are optional; app functions without these.
- **Future Cloud Sync**: Privacy policy should be clear on data collection and sharing.

28.4 Permissions

Currently requested:

- **INTERNET**: Required for Firebase (future) and Anthropic API.
- **RECORD_AUDIO**: Required for voice commands (future).
- **READ_EXTERNAL_STORAGE**: Required for pattern images (future).

All permissions follow Android's runtime permission model (API 23+).

29 Known Limitations

29.1 Compilation State

Critical: The application does not compile due to unresolved references and parameter mismatches in multiple modules. This must be resolved before any functional testing or deployment.

29.2 Firebase Configuration

Limitation: Cloud features are non-functional until 'google-services.json' is configured. Users are limited to local-only operation.

Impact: No cross-device sync, no backup, no cloud authentication.

29.3 Voice Commands

Limitation: Voice module is not integrated with UI screens. Voice commands are not functional.

Impact: Users must use manual button controls. Hands-free operation not available.

29.4 Stitch Library

Limitation: Feature directory exists but no implementation. Stitch library feature is not available.

Impact: Users cannot reference visual stitch guides within the app; they must consult external resources.

29.5 Notifications

Limitation: Notification infrastructure not implemented. Daily reminders and project pause alerts are not sent.

Impact: Users rely on self-initiated app usage; no external encouragement.

29.6 Internationalization

Limitation: Many UI strings are hardcoded in Portuguese. Non-Portuguese speakers encounter mixed-language interfaces.

Impact: App usability reduced for international audiences.

29.7 Pattern Import/Export

Limitation: No mechanism to import patterns from external sources (e.g., Ravelry, user-created PDFs).

Impact: Only seeded patterns and manually entered patterns are available.